Practical DevOps Project Portfolio (PORT1)

Enterprise Hospital Management System (HMS)

December 10, 20XX

—

English Literature & Composition

—

Professor Graves

Section 1 : Project Initiation & Development Strategy

1.1. Introduction & Project Scope :

The Enterprise Hospital Management System (HMS) is a web-based application composed of multiple layers designed to streamline clinical work processes, reduce administrative workloads, and manage information with an emphasis on security. A modern clinical setting cannot effectively manage patient care delivery without optimizing its work processes. Digitizing the paper-based clinical process enhances the accuracy of the records, conforms with established medical tracking protocols, and efficiently facilitates inter-departmental communication between different hospital units. The Enterprise HMS is able to secure sensitive patient records through role-based access privileges and maintains complete audit trails. The following areas are included in the project and are derived from structural system modules to include seven integrated core operational areas:

Patient Management Controls the registration, profiling and lifecycle updates of patient records *(Mir, 2026a)*

Employee Management Consolidates all clinical and administrative staff's administrative tracking, common structural scheme and role position by a role attribute which dynamically assigned *(Mir, 2026a).*

Medicine Management Records volumes and inventory value parameters for pharmaceutical storage *(Mir, 2026a).*

Bed Management Manages structural ward logistic, the bed occupancy in real time and the flow and assignment of beds *(Mir, 2026a)*

Attendance Management Checks employees' arriving and leaving time, and the timing of schedule *(Mir, 2026a).*

Task Management Works as a secure, internal communication channel to allocate clinical responsibilities and monitor workload *(Mir, 2026a).*

Billing Management Calculate, record and trace clinical invoice production based on services provided *(Mir, 2026a).*

1.2 SDLC Methodology Selection & Justification :

Software engineering operations relies heavily on 3 parameters: project type, time limitation and the underlying technical risks. Traditional linear, sequential Waterfall model (where every phase must be completed to absolute satisfaction before proceeding to the next phase) of development would bear serious risks of deployment in the context of HMS platform. While Waterfall model is ideal for safety-critical system or simple linear transaction scripts or anything where requirement changes can be

minimal, it is not ideal for collaborative rapid web application, where integration must be ongoing *(Kruchten, 2004, p.31).*

Therefore, Agile Methodology was adopted as operation framework. Agile decomposes architecture into smaller bits, where development can be performed in short cycle termed 'Sprint' in order to produce software increment after every Sprint.

This concept of structural design is illustrated in the Mona Lisa Analogy described in class. In the standard Waterfall model the user would list all user requirements first and the programmer would proceed alone and only show the painting at the end. Should the user not like the painting when completed, all development time is wasted. In the Agile model the developer creates the software in very small chunks and after each iteration (sprint) a working version of the software can be viewed by stakeholders and feedback given. The software is continually refined throughout the project thus ensuring that at the time of deployment there is minimal chaos and maximum stability *(Mir, 2026a).*

1.3 Implementation of the Six SDLC Phases :

The life cycle of development for HMS follows an ordered approach through the six basic engineering phases: Poor Rabbit Danced During The Dark Moon:

Planning (Poor): Project initiation baselining, scheduling resources through Gantt charts, creating the critical path and building prospective risk evaluation logs.

Requirements Gathering (Rabbit): Create 24 separate functional requirements formal documentation in each clinical domain to ensure all logical gaps are closed prior to code being written.

Design (Danced): Creation of formal architectural diagrams which may include structural UML Class diagrams, Entity Relationship Diagrams (ERD), dynamic sequence flows, component diagrams.

Development (During): Raw and abstract visual design elements to a well-organized set of modular Python source code, written in the VS Code IDE, under a minimal framework of web development: Flask *(Mir, 2026b).*

Testing (The): Systematically check the behavior of system via precise testing workflows, verifying constraints will validate without the loss of data or records being dropped *(Mir, 2026b).*

Maintenance & Deployment (Dark Moon): Safely shipping validated software builds to a live cloud platform, resolving version control deltas, and documenting system behavior for future releases.

Section 2 : Project Planning & Risk Engineering

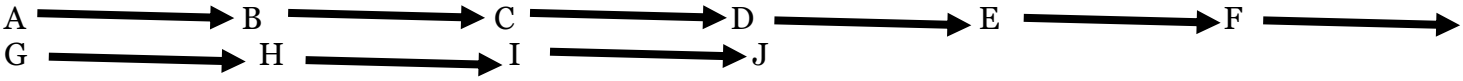
2.1 Project Task Dependencies Table :

To ensure a precise and measurable schedule and to make the resource limitation obvious, the development process of the HMS was split into ten engineering activities. With respect to distributed software development environments, the proper management of predecessors in the activity network allows developers to work concurrently without branch merging conflicts or data loss due to erroring out CI *(Brooks, 1995, p.21)*. Every module - i.e. From patient information manipulation, through ward management logic and on to automatic billing generation - is strictly dependent on its structural predecessors being already defined. E.g. Database schema cannot be modeled before the specification of the use cases' scopes, actors and class matrices are completely locked. The dependencies table below represents the entire technical activity flow within the constraints imposed by the unified 10-day implementation phase of the joint team scrum planning:

Task ID	Task Description	Immediate Predecessors	Estimated Duration (Days)
A	Requirements Elicitation & Domain Analysis	None	3
B	Use Case Responsibility Matrix & Formal Specifications	A	2
C	Visualizing Structural Architecture (UML Use Case & Class Diagram)	B	3
D	Database Schema Normalization & ERD Mapping	C	2
E	Layout Design & Behavioral Modeling (Sequence & Component Views)	D	2
F	Base UI Layout & Navigation Implementation (base.html)	E	3
G	Backend Database Integration & Model Layer Code Generation	F	4
H	Core Module Business Logic Development (Routes & Service Layers)	G	10
I	Multi-Role Security, Validation, & Alpha Integration Testing	H	3
J	DevOps Pipeline Optimization & Cloud Deployment Configuration	I	2

2.2 CPM Network Diagram & Gantt Chart Analysis :

After the Dependencies Table (revised) structural information is fed into a full CPM mathematical pass, it has isolated the one absolute longest series of interdependent software development. This network shows that it does not have float throughout the principal nodes and thus provides the real framework timeline of



The real lowest possible duration is 34 days, this gives a basis by which each of the mentioned tasks in sequence directly affects final delivery of the portfolio as a delay. The network has showed that a minor change of backend architecture, or a last minute refactor to the database schema increases the execution time significantly *(Brooks, 1995, p.87).* Figure 1 shows the timeline across a 12-week deliverable by breaking the workflow down into sections of timed activity and ensuring that it does not affect any team member's ability to effectively allocate their resources to each of these sections of work matching the Scrum constraints.

2.3 Formal Project Risk Assessment Log :

Risk management is a systemic engineering practice for identifying, analyzing, and mitigating against detrimental events from causing instability to the project and chaos in the teamwork process across project cycles. The current web applications, risk analysis helps safeguard the integrity of systems boundaries by avoiding runtime data from falling and failure to manage configuration file entries. The matrix shown below is a record of the risk management official logs of HMS project (Mir, 2026b):

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (P x I)	Proactive Mitigation Strategy
R-01	Scope Creep / Missing Module FieldsLate identification of mandatory requirements.	2	4	8	Establish a frozen requirements baseline using a verified Use Case Responsibility Matrix before writing code.
R-02	Merge Conflicts / Overwritten FilesTeammates modifying identical code segments simultaneously.	4	3	12	Implement strict branch isolation rules; assign isolated files per module and mandate a git pull before starting work.
R-03	Database Constraint DroppingForeign key constraint violations in SQLite during testing.	3	4	12	Enforce a strict OOPS mapping design pattern; utilize SQLAlchemy

					database models to handle data references automatically.
R-04	UI InconsistencyNavigation links breaking or rendering incorrectly across different views.	3	3	9	Centralize the template layout within a master base.html architecture using Jinja2 inheritance blocks.

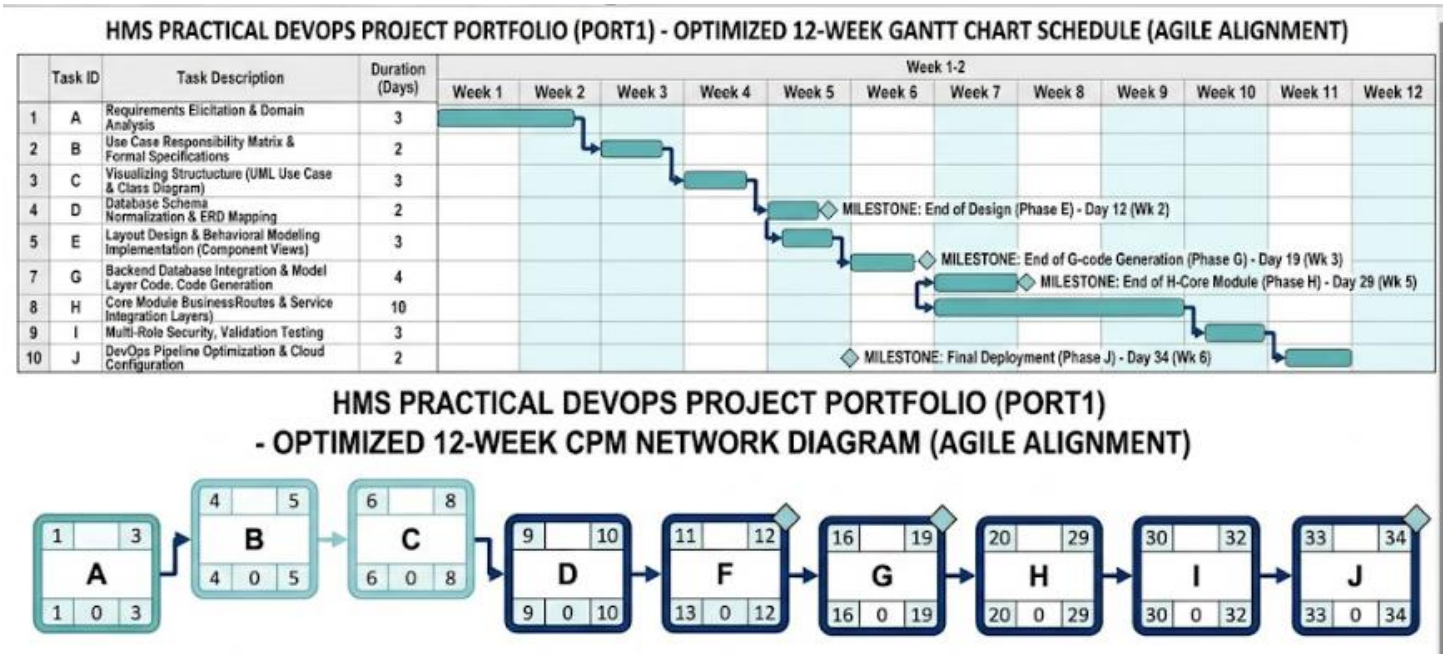


Figure 1 : Gantt chart & CPM for hospital management system.

Section 3: Requirements Engineering

3.1 Functional Requirements Specification :

Requirements engineering is the analytical back-end of the system boundary and will gather all stakeholder expectations systematically before software engineering teams carry out the development process. Functional requirements define the exact technical operations, calculations constraints and data flows which must be implemented in the web platform. The Hospital Management System (HMS) requirements definition listed 24 distinct functional requirements over the system's functional areas. These requirements have been structured into the seven middle-layer components (for a clean separation of implementation) as follow:

Patient Management Module: FR-1.1: The system must allow registered administrative personnel to input and store new patient information, consisting of a unique identifier, full name, age, contact telephone number, and home address.
FR-1.2: The system must be able to update selected fields for existing patient entries.
FR-1.3: The system must allow authenticated personnel to list all patient records available. Employee

Management Module: **FR-2.1:** The system must allow administrative personnel to register clinical and administrative staff employees, storing information such as employee unique identifiers, names, contact channels, and current positional rights.
FR-2.2: The system must allow authorized staff to modify employee roles dynamically.
FR-2.3: The system must be capable of disactivating orarchiving employee accounts in a secure manner.
Medicine Management Module: **FR-3.1**: The system must keep an inventory of medicines in the pharmacy, tracking each medicine via medicine unique identifiers, generic names, current counts and unit cost.
FR-3.2: The system must be able to subtract quantities from medicine inventory when they are dispensed to patients.
FR-3.3: The system must be able to raise alerts when a particular medicine is low in stock. Bed

Management Module: FR-4.1: The system must reflect the state (occupied/available) of all individual hospital beds and be capable of tracking beds by their unique identifying numbers.
FR-4.2: The system must associate a patient record with a currently available bed, transitioning it to the occupied state.
FR-4.3: The system must transition available beds to the free state upon patient discharge.

Attendance Management Module: **FR-5.1:** The system must allow employees to record check-in times using unique worker codes at given access points.
FR-5.2: The system must persist recorded time stamps and the presence status at each shift into its data storage.
FR-5.3: The system must calculate employee punctuality reports for monthly cycles. Task

Management Module: FR-6.1: The system must allow registered administrative personnel to generate and assign specific clinical tasks to employees based on worker ID.
FR-6.2: The system must store the exact text description for the assigned tasks and its current status (pending/completed/etc).
FR-6.3: The system must allow workers to update task completion status dynamicly. Billing

Management Module: FR-7.1: The system must calculate the total amounts owed to the hospital for a specific patient based on collected medical records.

FR-7.2: The system must produce formatted receipts showing the client's invoice ID, total charges and date of payment.
FR-7.3: The system must keep track of the status of paid bills.

3.2 Non-Functional Requirements Specification :

Non-functional requirements defines the operating quality requirements, safety limitations and technology restraints that ensure the integrity of the system. Non-functional requirements are the parameters under which the web architecture should run while live in production, unlike functional characteristics.

The strict operational guidelines that will ensure a professional web architecture design baseline are:

Security & Data Privacy - Limit data access thresholds through multi-role privileges and allow only the relevant data to be visible by clinical users, also ensure sensitive database transactions must occur over a secure channel to protect against unwanted intruders *(Mir, 2026b).*

Performance & Latency - Each light Flask controller and SQLAlchemy model transaction must process query at less than 2.0 seconds under a standard concurrent operating load to ensure clinical users are not kept waiting when during a clinical emergency *(Nielsen, 1993, p.135).*

Reliability & Availability - Each SQLite relational database engine must not drop structure with a structural availability of at least 99.9% during operating hours and allow system transactions safely through it.

Maintainability & Scalability - The system architecture's code must strictly adhere to a separated 3-layer design pattern (Controller, Service, Model) so as to allow for easier modular incrementation and schema logic changes.

3.3 Use Case Description: Assign Bed to Patient :

To logically connect requirements analysis to visual UML structural behavior modeling, core system functionality needs to be specified structurally through use case specifications. The block of text below contains pre-conditions, a transaction, and error handling specifications for the assigning a patient to a bed activity:
Use Case ID:UC-BM-04

Use Case Name: Assign Available Bed to Patient

Primary Actor: Administrative Staff

Description: Allows the administrative staff to use the interface, select an open bed row and bind that row to an active profile record for the patient.

Pre-conditions: administrative staff is logged in, a patient record already exists in the database, one ward bed status attribute must be available.

Post-conditions: one ward bed status attribute must change to occupied, unique patient key added to the database table for the bed, and interface reflects successfully assigned bed.

Main Flow of Events (Chronological Path):

1. Actor issues 'Open Assign Bed Page' command route in web navigation interface.
2. Interface queries system controller for available structural listings.
3. Controller queries bed table for vacant rows and renders them to the actor.
4. Actor enter patient's unique identification code and desired vacant row.
5. Actor submits assignments data to begin validation routine.
6. System verifies integrity of data and updates desired row's bed status flag to occupied in storage.
7. System adds patient's code as a foreign key constraint to lock the mapping.
8. Database returns a successful affirmation delta to the central service.
9. System sends valid response to the interface portal for rendering.
10. Interface presents success affirmation message.

Alternative Flow (Exceptions): System finds that an invalid patient ID is supplied in the entry; System aborts the operation transaction and fires the warning: "Invalid Patient ID - Allocation Rejected."

3.4 Use Case Responsibility Matrix (UCRM) :

The Use Case Responsibility Matrix. It is used as an acceptance check to establish system boundary accesses, ensuring that only designated actors initiate the functionality requirements. The mapping provides system security acceptance prior to class logic development. Access by function by operational role can be defined as follows:

Use Case Description	Administrative Staff	Doctors	Nurses	Pharmacy Staff	Finance & Support Staff
UC-1.1 Register New Patient	X	—	—	—	—
UC-1.2 Update Patient Logs	X	X	X	—	—
UC-4.2 Assign Ward Bed	X	—	—	—	—
UC-3.2 Update Medicine Inventory	—	—	—	X	—
UC-5.1 Record Daily Attendance	X	X	X	X	X
UC-6.1 Assign Clinical Tasks	X	X	—	—	—
UC-7.2 Generate Client Invoice	—	—	—	—	X

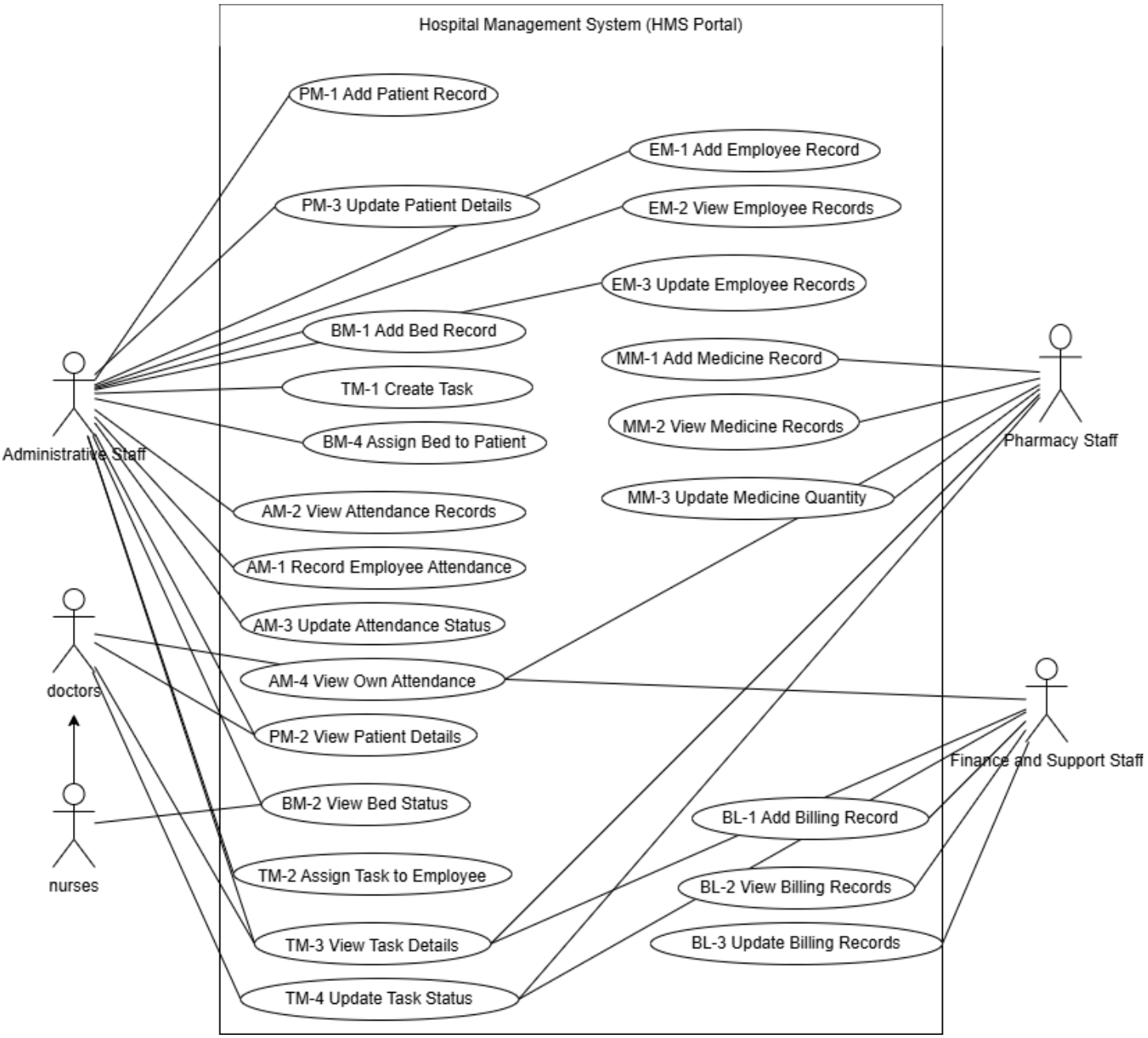


Figure 2 : use case diagram for hospital management system.

Section 4: System Architecture & Structural Design

4.1 Object-Oriented Design & Class Derivation :

Starting from scattered behavior descriptions, object-oriented design builds a clear backbone turning those inputs into dependable, repeatable code structures. Through careful analysis, development teams extract class outlines, data traits, and access rules by tracing steps laid out in earlier use scenarios. Within hospital software settings, real-world elements like medical records, staff codes, or inventory levels become fixed features inside matching program units.
Still, operations like setting up a patient room, generating assignments, or working out billing figures appear as public method definitions within classes. These boundaries let every domain handle its internal processes without interference. Unauthorized adjustments across components get blocked. Changes move only through checked control routes *(Fowler, 2004, p.43).*

4.2 Architectural Structural Review (Option 1 vs. Option 2) :

The overall architectural design of the HMS is expressed using a well-defined UML Class Diagram, which explicitly identifies the class boundaries, data members and visibility of functions. Seven main related classes (matching the blueprints for backend operation) form the backbone of the system, including Patient, Employee, Medicine, Bed, Attendance, Task and Billing. In order to strictly enforce type-safety, the attributes are all defined with the precise data types required, and methods are specified with the precise function signature. Objects communicate via clearly labeled directed association arrows, and multiplicity constraints are defined stringently to represent this (*Fowler, 2004, p.68)*; for example, to represent the way a patient may accumulate many historic invoice documents throughout their care history, a 1-to-many relationship is established between patient and billing. Conversely, to represent the fact that bed allocation is optional for any given patient at any given point in time, the multiplicity of a patient to bed relationship is enforced to 1-to-optional *(Fowler, 2004, p.68).*

4.4 Backend Relational Database Design (ERD) :

Finally, the backend relational database layer. This layer translates the high-level object oriented class diagrams into flat relational tables highly tuned for the SQLite data engine. Class diagrams outline states and program methods, whereas our Entity Relationship Diagram outlines only the data that will be serialized and ignores any methods in order to isolate tables and associated primary and foreign keys. Tables of participation states among entities should be ordered in ways that permit or deny certain records to avoid data loss under production traffic conditions *(Elmasri and Navathe, 2021, p.74).* Table linkage will be rigidly enforced through relational rules alone; that is Bed.patientid

identifies occupant(s) of a bed, Billing.patientid tracks who is being billed, Attendance.employeeid and Task.employeeid will link the logged hours or tasks assigned to our staff object. Each constraint will be specified with one of theSQLite datatypes such asINTEGER (for autoincremented keys),VARCHAR(for textual fields) orDATETIME (for timestamps) so that data is never lost during runtime execution of services *(Elmasri and Navathe, 2021, p.182).*

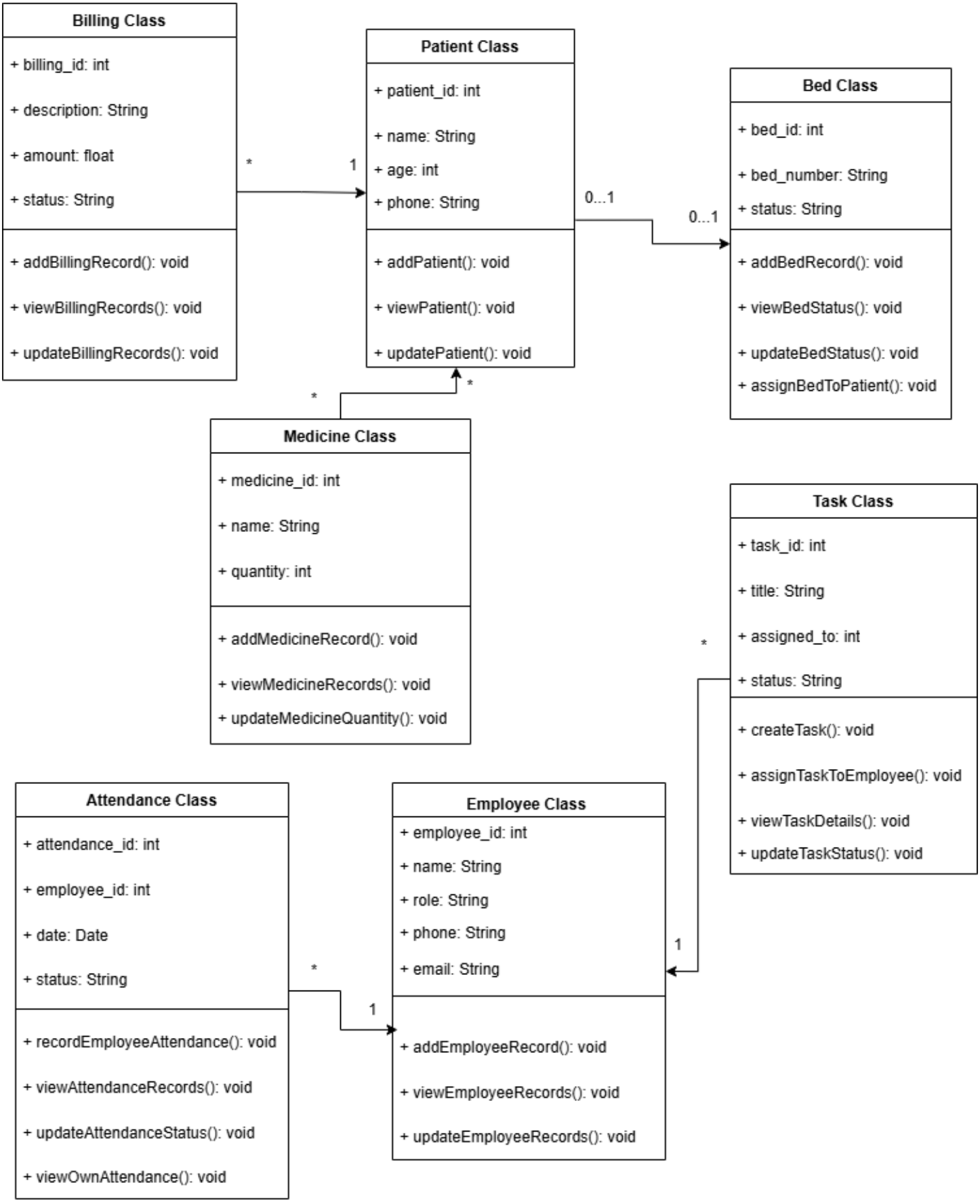


Figure 3 : Uml class diagram for hospital management system.

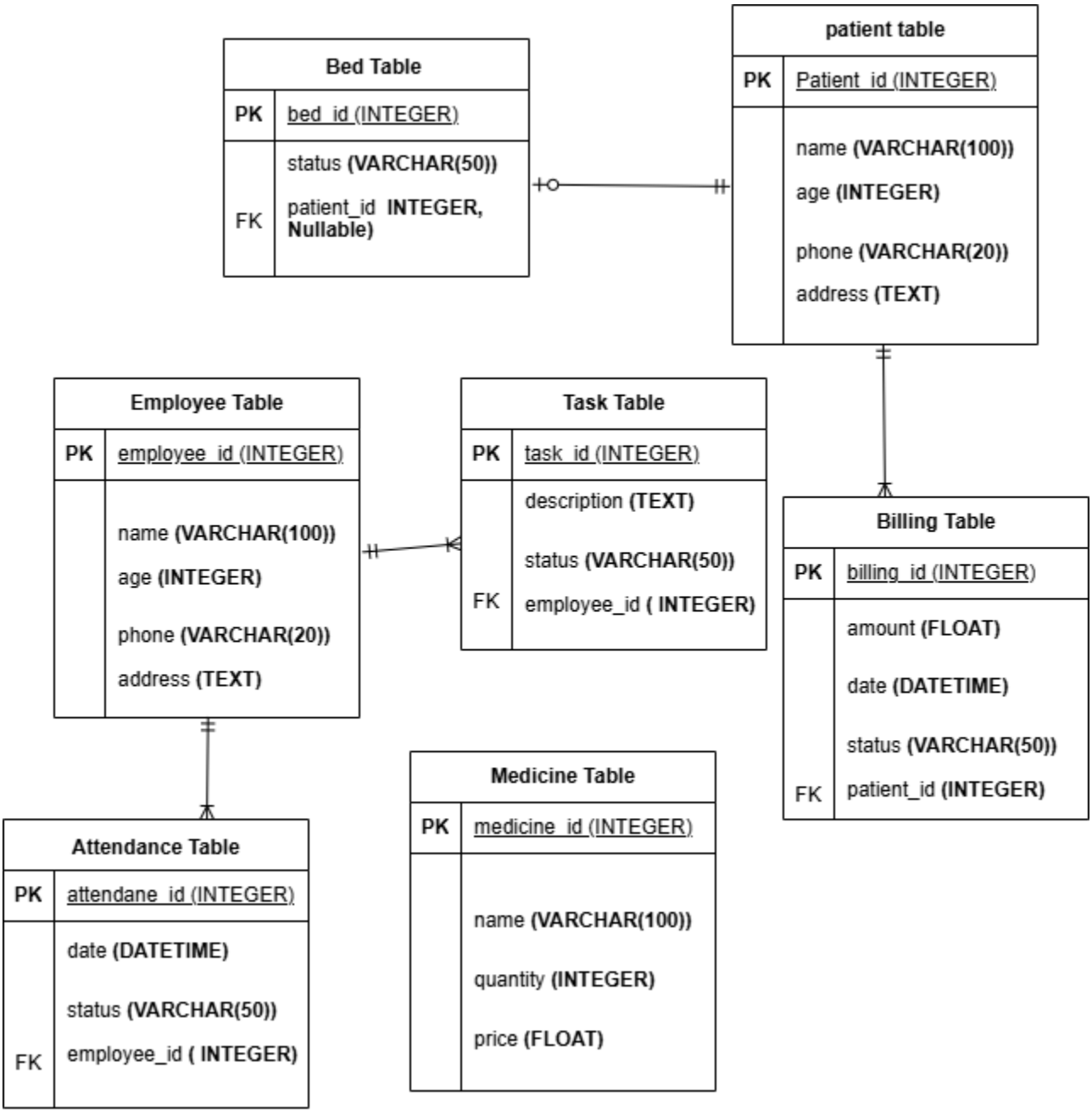


Figure 4 : Uml Entity Relationship Diagram for hospital management system.

Section 5: Behavioral & Component Architecture

5.1 Level-1 System Behavioral Modeling :

Level-1 behavior modeling describes system's time-based interactions at runtime. Structure class layouts define static boundaries while behavioral modeling diagrams how runtime elements collaborate to implement a given functional use case. When it comes to designing efficient web applications, modeling of such interactions would determine when each element transmits a message to which, helping to identify communication latencies and ensuring that the backend correctly validates that the execution logic itself runs, before committing the changes to persistent store. By observing sequence of messages between various layers of a system, the engineer can reliably pinpoint execution bottlenecks and get rid of message dropping and transaction redundancy. HMS behavioral model formally defines sequences in which the HMS implementation runs, e.g., authenticating users, adding/removing medicines in hospital, assigning rooms/equipment in the hospital etc, guaranteeing error-free operation when running on real-life clinical loads.

5.2 UML Sequence Diagram Specifications :

Runtime behavior dynamics of HMS is precisely detailed using a formal UML Sequence Diagram artifact, illustrating the timing of messaging within the loop to fulfill UC-BM-04-bed allocation process flow. The diagram traces a sequential message flow along a vertical four-tiered lifeline structure:
Actor → User Interface → System Controller →Relational Database, starting when Administrative Staff submits an allocation request in a web page (browser). The User Interface element is triggered to submit a synchronous POST request (over HTTP) to a Flask central Controller blueprint. The central Controller immediately delegates responsibility to the Python service layer to enforce integrity constraints for the corresponding entities; that the patient to allocate is indeed present and the queried Bed record row is currently in a status of 'available.' If integrity is maintained, the service layer will directly persist an updated execution delta to the underlying SQLite data layer. The SQLite layer will enforce foreign-key constraints and will subsequently lock the target data layer row and report a successful persistence back up the lifespan of the request; to the Administrative Staff in a simple notification message.

5.3 High-Level Modular System Boundaries:

The top-level modular system boundary isolates the layers of an enterprise application to achieve low coupling and high structural cohesion. Segmenting an application with tightly integrated modules into clean operational layers helps avoid design chaos and ensure the modification within a module doesn't cause other unintended failures. For the HMS application, a strictly defined 3-layer architecture was opted for, dividing layers by handling user input and presenting data to them (Presentation Layer), implementing business logic and access permission control (Application/Service Layer), and storing and retrieving data (Data Access Layer):

- Presentation Layer: Developed in HTML5 with Bootstrap template and Jinja2 template inheritance for rendering client side contents and receiving user input *(Nielsen, 1993, p.112).*

- **Application/Service Layer:** Driven by Flask routes and individual service modules to deal with business logic, access permission, and validate transaction operation.

- Data Access Layer: Configured with SQLite database and object relation mapping model built on SQLAlchemy to manage the data storage.

5.4 UML Component Diagram Architecture :

The internal physical code layout and runtime module coupling of the HMS web application are modeled by an elaborate UML Component Diagram. This class structural diagram portrays the relationship between each software file at the presentation, services and database layer boundaries. Given the strict specifications from the project requirement, the component architecture is designed and presented with detailed stereotyped interfaces, expressing the routes components are utilizing the back-end logic exposed by services module components. Four dashed lines, showing runtime dependencies between component usage, are also displayed on the diagram: routespatient, which depends on servicespatient, routesemployee on servicesemployee, routesbed on servicesbed, and routesattendance on servicesattendance. All the above components is linked down to a single app.py entry file which act as the central registration kernel for all flask blueprint modules and mapped the database coupling down to a persistent sqlite file *(Mir, 2026b).*

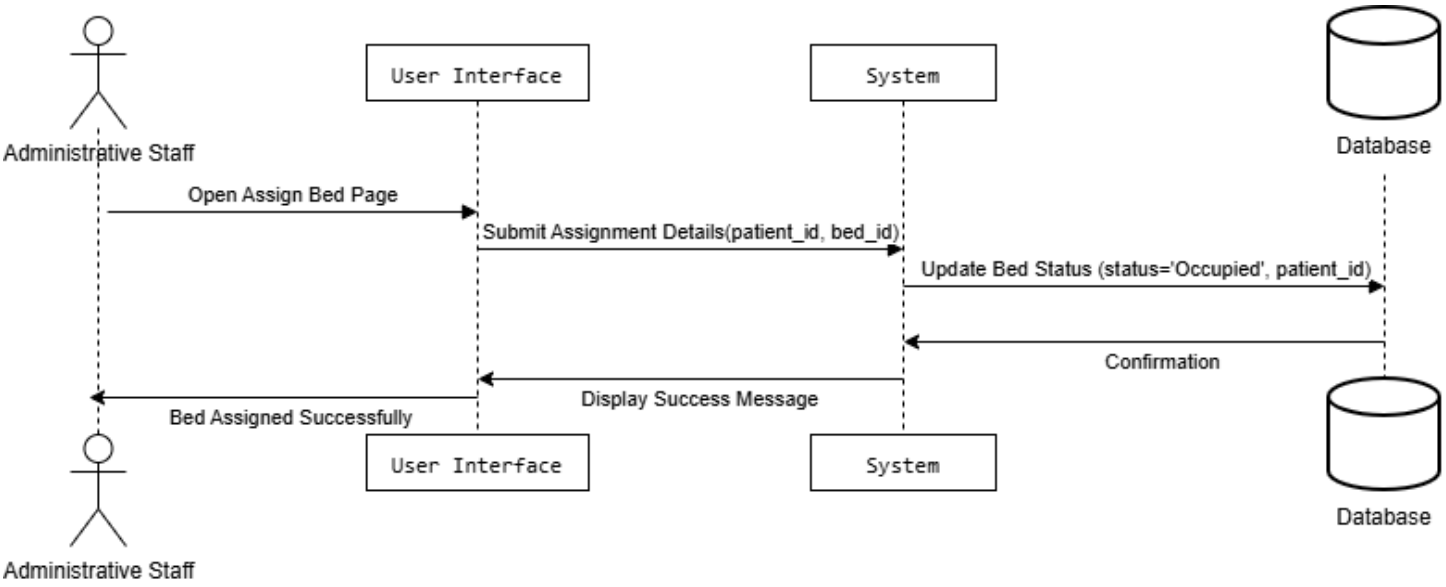


Figure 5 : Uml Sequence diagram for hospital management system.

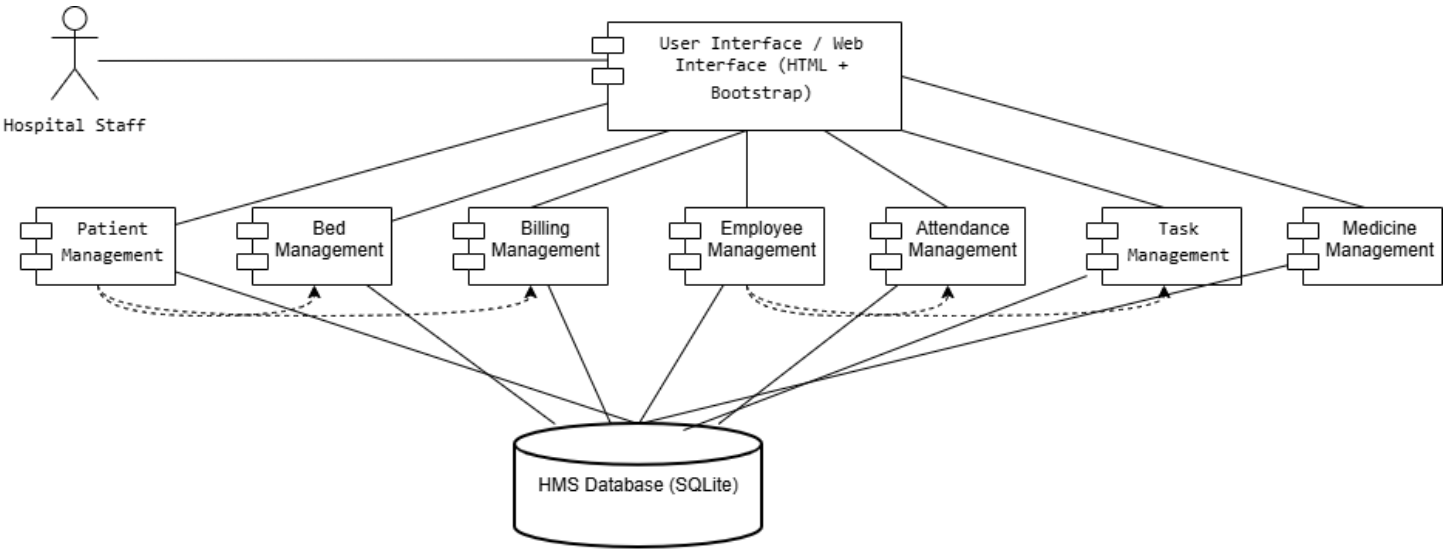


Figure 6 : Uml Component diagram for hospital management system.

Section 6: DevOps Engineering & Implementation Evidence

6.1 Git & GitHub Configuration Management :

Config Management and Version Control are the life-line of present day DevOps practices, that directly avoid collaboration disaster within the teams composed of multi-role software engineers. With the development of Hospital Management System(HMS), git was used as a central device to record every amendment, manage the sub-tree of the code and restrict every member of the team to be able to override an working file. A rigid feature-branching model was established so that fundamental system features can be completed in total seclusion. For example, structural elements like the workflow for patients and pharmaceutical tracking were partitioned onto separate branches such as module-medicine and feature/bed-management, before undergoing review processes. Automated git pull was executed by the programmers to merge upstream amendments and manage merge conflicts locally before the pushing of the local changes to the remote repository in order to assure the working capability of main branch after merge. Such kind of good practice for branching is evident through commit history: with prefixes like feat(UI): or fix(DB): used in commit message which describes delivery process of the project through descriptive Agile prefix.

6.2 The 3-Layer Implementation Flow :

Following a firm 3-layer architecture design pattern which strictly isolates user-interaction layer from business-logic rules layer and data-modeling-for-relational-storage layer allows for easy continuous delivery and maintainability of clean code. Following a design, for any single runtime transaction it passes through a very formal linear flow in architecture (code-to-architecture). It flows from a Flask Controller route to the object-mapped storage layer:

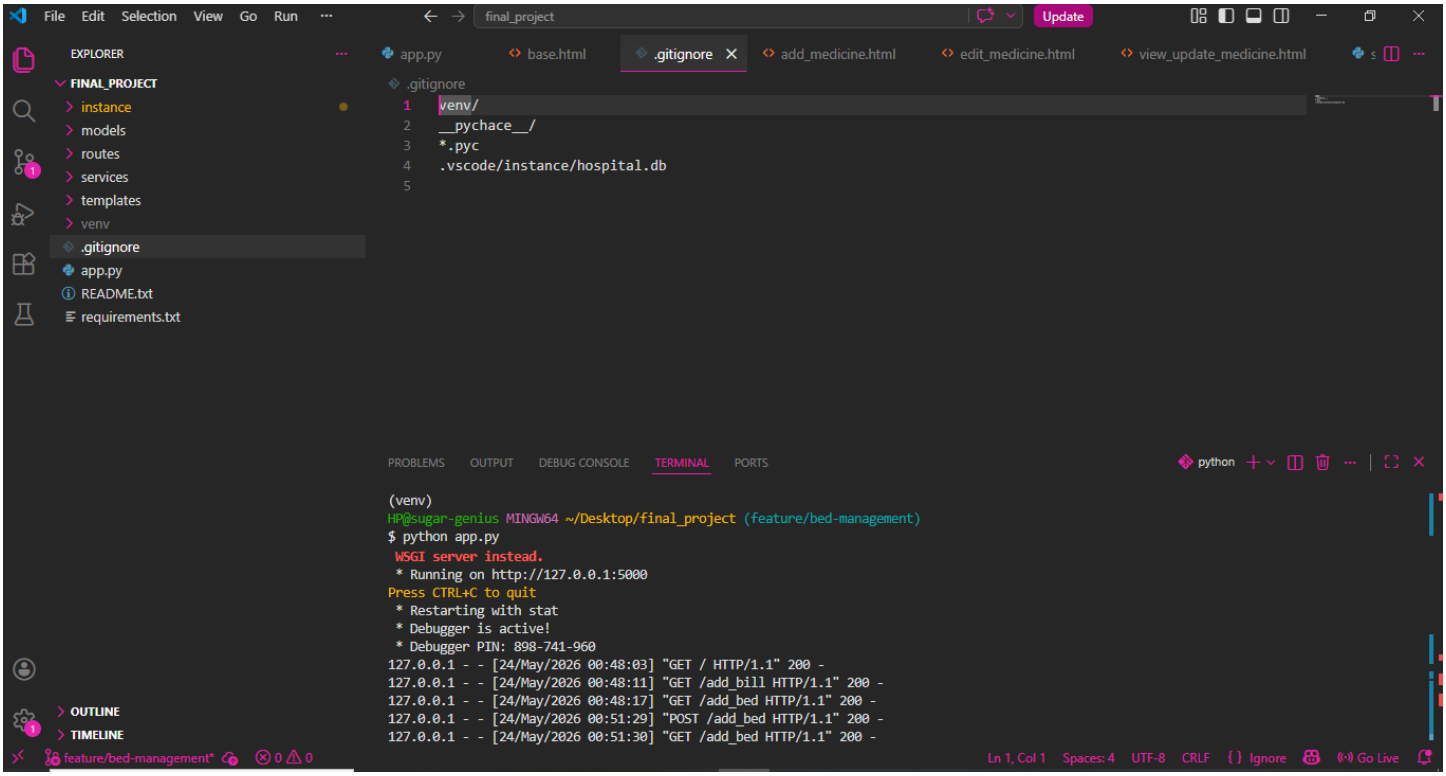
Presentation Layer(Flask Routes): The run cycle starts off with a file like routesbed.py, which provides a named endpoint such as @bed.route('/addbed', methods=['POST']) to handle user form data input parameters.

Application Layer(Services Engine): Then passing the data structures directly to the application service engine layer, for example service_bed.py to perform condition check, validation, roles check, etc based on incoming inputs.

Data Access Layer(SQLAlchemy Models): When the check passed, then pass to the data access layer at models.py where create the data class (e.g class Bed(db.Model)), generate an object based on input parameters and assign it to the object parameters and add a secure database transaction to write directly to the database files (SQLite).

6.3 Testing Evidence & Portfolio Summary :

This last section in the DevOps pipeline requires detailed automated and manual checks in order to guarantee system stability, trace software craftsmanship level, and assure all use case parameters meet the original client requirement. Testing proof has been incrementally collected across the individual Alpha loops of integration, to assure the proper existence of database constraints, testing foreign key cascades and testing that dynamic dropdown lists appear in master layout. Using a combination of artifactive planning docs, functional spec, OO structural designs, and clear CM proof, this portfolio exhibits an industrial level of production software engineering: The complete working Hospital Management System code, has been securely pushed to the distant GitHub origin, and all functional behavioral proof has been recorded through an interactive video demo, walk through.



Hospital Management System

Patient | Employee | Medicine | PatientBed | Billing | Attendance | Task

Welcome to Bed Module - Add Bed

Bed Number / Identifier

33

Assigned Department

IT Department

Initial Status

Occupied

Save

Hospital Management System

Patient | Employee | Medicine | PatientBed | Billing | Attendance | Task

Welcome to Bed Module - View & Update Beds

ID	Bed Number	Department	Status	Actions
1	33	IT Department	Occupied	Edit/Update

REFERENCES

Brooks, F.P., 1995. The Mythical Man-Month: Essays on Software Engineering. Anniversary ed. Boston: Addison-Wesley.

Elmasri, R. and Navathe, S.B., 2021. Fundamentals of Database Systems. 7th ed. Boston: Pearson.

Fowler, M., 2004. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed. Boston: Addison-Wesley.

Kruchten, P., 2004. The Rational Unified Process: An Introduction. 3rd ed. Boston: Addison-Wesley.

Mir, I., 2026a. Agile Development and DevOps [SEN5002]: Introduction to SDLC and Systems Requirements. Lecture Slides, Module 1. Kenitra: SIST School of Software Engineering.

Mir, I., 2026b. Agile Development and DevOps [SEN5002]: Architectural Frameworks and Component Designs. Lecture Slides, Module 5. Kenitra: SIST School of Software Engineering.

Nielsen, J., 1993. Usability Engineering. Boston: AP Professional.